CSAW 2026 · AI HARDWARE ATTACK CHALLENGE (QUALIFIER)

An AI-generated hardware Trojan in a recovered iCE40 crypto core

Bitstream recovery → verified baseline → Trojan → simulated exploit

Vaughn — NYU Design Project (solo)

AI collaborators: GPT-6.1 (recovery) · Claude Opus 4.8 (baseline, Trojan, exploit)

Tools: OSS CAD Suite (IceStorm, Yosys, Icarus Verilog), Python

2026-11-06

FINDINGS OVERVIEW

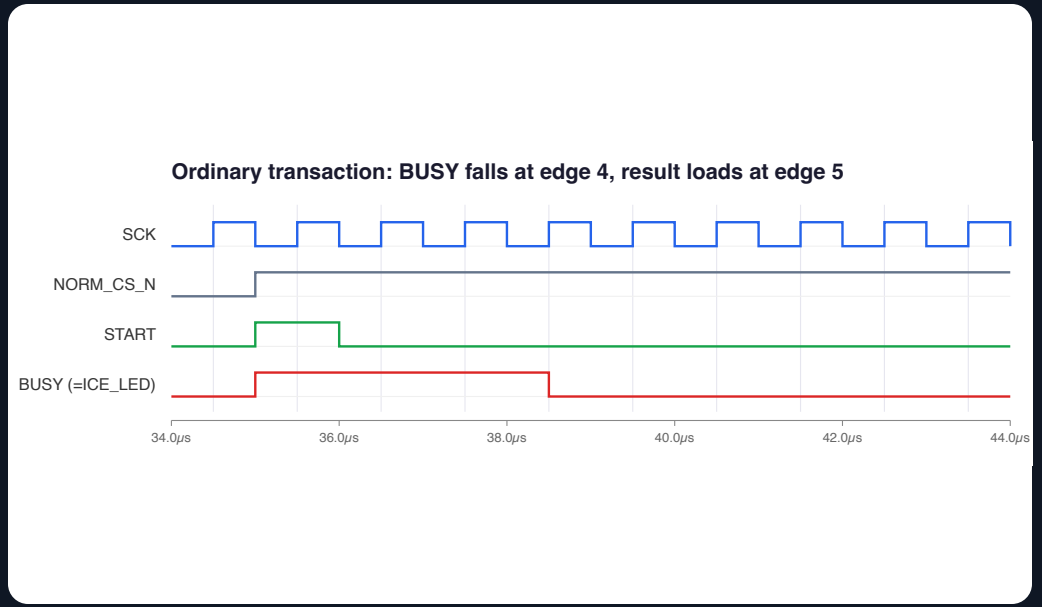
Project goal and what we found

- Goal: recover the supplied FPGA crypto IP, verify it, then have AI insert and exploit one hardware Trojan.
- Bitstream recovered; cipher reconstructed; self-checking baseline passes 127/127.
- AI-generated Trojan + exploit: 140/140 simulation checks; waveforms + resource diff captured.
- Confirmed end-to-end on the real Hackster board: dormant in normal use, payload fires on cue.
- One reproducible command regenerates every simulation result.

WHAT IT DOES

Accelerator architecture & ordinary transaction

- 32-bit, 4-round block cipher. Raw SPI peripheral + sideband control (no AXI/Wishbone).
- Unified clock: SCK drives both the SPI shift register and the crypto engine.
- Transaction: reset → shift 32 bits in (CS low) → pulse START → wait → shift 32 bits out.
- BUSY high through edges 1-3, low from edge 4; result loads on edge 5.
- ENC_DEC: 0 = encrypt, 1 = decrypt. BUSY is mirrored on ICE_LED.



REVERSE ENGINEERING

Bitstream recovery and open tools

- iceunpack bitstream → .asc (device 5k); icebox_vlog → structural Verilog (~3.2k lines, 9 ports, 4 block RAMs).
- recover.py is deterministic: records input SHA-256 + exact commands; asserts the circuit body is byte-identical.
- Only edit: ANSI→non-ANSI scalar port headers so Icarus accepts the netlist. Memory primitives via cells_sim.v.
- Open tools only: OSS CAD Suite — IceStorm, Yosys, Icarus Verilog — plus Python. No vendor tools.
- Supplied vs independent: bitstream + datasheet + solution PDF are supplied; the netlist + analysis are ours.

TRUSTWORTHY BASELINE

Baseline verification & resolved naming question

- Independent Python reference model of the two PDF round functions (AES S-box, K=0x8E043CB1, 4 rounds).
- Self-checking bench vs the recovered netlist: 60 inputs x 2 modes = 120/120; round-trip + reset + back-to-back.
- Resolved the open question: NO mode/function swap. mode0 = PDF encrypt, mode1 = PDF decrypt — names agree.
- The earlier 'swap' came from a mis-transcribed software cipher (0x137B1F72 is really the decrypt output).
- Measured timing: minimum safe readback = 5 SCK edges, not 4 (BUSY-low ≠ result ready).

AI USAGE

AI inputs, generate–test–revise, guardrails

- Inputs to the AI: recovered netlist, the datasheet, the PDF's S-box/key/round functions, observed vectors.
- Generate-test-revise is logged. Real revision: first reference model failed all 4 vectors → fixed the mode mapping → passed.
- All RTL, testbenches, scripts, and waveplots are AI-generated; the human reviews, runs, and accepts.
- Guardrails — honest account: no refusal and no bypass. In a sanctioned academic context the model built the Trojan directly.
- Only 'guardrail-like' behavior was voluntary: narrow scope, Trojan logic labeled, stealth stated as detectable.

THE TROJAN

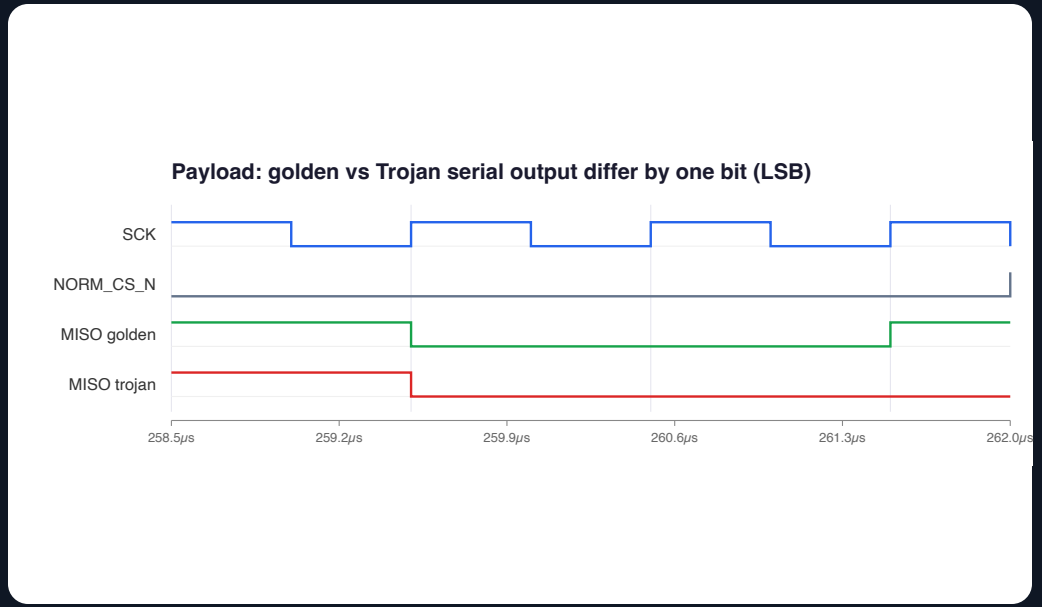
Trigger, payload, insertion point & taxonomy

- Insertion: design-time, RTL source. Diff crypto_core_clean.v vs crypto_core_trojan.v = the whole change.
- Trigger: digital, input-controlled, SEQUENTIAL — rare ordered 3-word sequence 13375EED → C0DEFACE → A5A5A5A5.
- Any wrong completed transaction resets progress; a word = SEQ0 restarts it; RST_N dis-arms.
- Payload: functional corruption, INTEGRITY — invert output bit 0 of exactly ONE result.
- Channel: the EXISTING SPI output — no added ports. Added state: 32-bit input capture + 2-bit counter.

EVIDENCE

Simulation and on-board evidence

- Dormant: 120 ordinary vectors bit-identical to the golden core. Fires: 0x4f92e7f9 → 0x4f92e7f8 (bit 0).
- On the real Hackster board: ordinary test still passes; armed SEQ2 returns 4f92e7f8; recovers after.
- Near misses, reset dis-arm, one-result scope, re-arm: all correct — 140/140 in sim.
- Stealth is MEASURED: +96 cells (+3.4%), +34 flops — not area-invisible; a known-answer test exposes it.
- Coverage is extensive but not a universal-equivalence proof (seeds + counts recorded).



WRAP-UP

Solo responsibilities & next steps

- Solo project. Human: requirements, scope, board, running/reviewing/accepting all code, this talk.
- AI collaborators do the hardware code: GPT-6.1 (recovery + first probe), Claude Opus 4.8 (baseline, Trojan, exploit, slides).
- All hardware/exploit code is AI-generated; full prompts, code, and test feedback are kept in ai/.
- Next step: power side-channel (CPA) to recover the key independently — board exposes ICE_PWR + ICE_TRIG.
- Then the finals' physical challenges. Deliverables on GitHub: RTL, testbenches, AI logs, brief, deck + PDF.